EDUCONNECT AI

Complete Project Build Architecture & Antigravity Implementation Guide

Tagline: Find the right people. Build the right school network.

Assignment: EDXSO AI Intern Assignment — LinkedIn School Influencer Discovery Tool

Project type: AI-powered professional profile discovery, semantic matching and relevance ranking MVP

Recommended repository: educonnect-ai

Primary stack: React + TypeScript · FastAPI · Sentence Transformers · Supabase PostgreSQL + pgvector · Gemini

This document is a build blueprint. It specifies architecture, data flow, ranking methodology, project structure, implementation phases, deployment, testing and demo requirements. It is not a claim that the application has already been built or that any profiles have already been verified.

1. Executive summary

EduConnect AI takes one reference professional profile and discovers other professionals who may be relevant to the school education ecosystem. It analyses professional context, retrieves candidates from an authorised dataset, ranks them using semantic and structured signals, and explains each match with traceable evidence.

The target audience includes school consultants/advisors, former principals, school directors, owners and trustees, education entrepreneurs, school-network professionals, and experienced people working with multiple schools.

MVP success criteria

• Accept a reference profile through pasted profile text, a profile URL as a reference identifier, an existing database record, or CSV input.

• Analyse the reference profile into validated structured fields.

• Search an authorised, traceable dataset and return at least 20 relevant candidates for the demonstration profile.

• Use an actual embedding-based semantic similarity or ranking method, not only exact keyword matching.

• Show name, role, organisation, location, evidence indicators, score, explanation and source link when available.

• Support filtering, sorting, saving candidates, discovery history and CSV export.

• Deploy the frontend and backend, publish source code and provide a 5–10 minute demo plus technical note.

Important: a URL pasted by a user does not grant permission to scrape LinkedIn. Do not bypass authentication, CAPTCHA, access controls or platform restrictions. Use authorised APIs where available, user-provided text, permitted public sources or a clearly labelled test dataset.

2. Product identity and GitHub setup

Item	Recommendation
Product name	EduConnect AI
Folder name	educonnect-ai
GitHub repository	educonnect-ai
Tagline	Find the right people. Build the right school network.
GitHub description	AI-powered school influencer discovery platform that analyses reference profiles, discovers similar education professionals, and ranks them using semantic similarity, professional experience, and school ecosystem relevance.
Topics	artificial-intelligence, python, fastapi, react, education-tech, semantic-search, sentence-transformers, pgvector, profile-discovery

Product positioning

Position it as an education-network intelligence platform, not a LinkedIn scraper. Its differentiator is explainable discovery: it identifies why a candidate is relevant based on professional context and source-backed evidence.

3. User journey and end-to-end workflow

Stage	System behaviour	User-visible output
1. Reference input	Accept pasted profile text, URL reference, CSV or saved profile	Reference profile submitted

Stage	System behaviour	User-visible output
2. Profile analysis	Gemini extracts structured professional attributes; Pydantic validates	Profile summary and extracted fields
3. Embedding	Sentence Transformer encodes profile text as a vector	Semantic representation generated
4. Candidate retrieval	pgvector similarity search plus metadata filters	Candidate pool retrieved
5. Ranking	Combine semantic, role, school-sector, network and location signals	Sorted ranked candidates
6. Explanation	Generate concise evidence-grounded match reasons	Why this profile matches
7. Save/export	Persist run, shortlist profiles and export CSV	Reusable outreach shortlist

Reference profile input

• LinkedIn URL: store as a profile reference. Do not assume that the application can fetch its contents.

• Profile text: allow the user to paste information they are authorised to provide.

• CSV import: validate required columns, show a preview and report rejected rows.

• Existing profile: allow selecting a previously imported profile from the database.

Profile fields to extract

• Name, current role, organisation and location.

• Years of experience, leadership seniority and career history when evidenced.

• School education experience and school-sector indicators.

• Consulting/advisory experience and multi-school exposure.

• Relevant professional summary, skills and evidence snippets.

• Source URL, source type, retrieval date and verification status.

4. Recommended technical architecture

Layer	Technology	Responsibility
Frontend	React, Vite, TypeScript	Dashboard, input forms, results, analytics, profile detail
UI	Tailwind CSS, shadcn/ui, Lucide	Consistent enterprise SaaS interface
Charts	Recharts	Role/location distribution and discovery analytics
Backend	Python, FastAPI, Pydantic	REST APIs, validation, orchestration and business logic
LLM	Gemini API	Structured profile extraction and evidence-grounded explanations
Embeddings	Sentence Transformers: all-MiniLM-L6-v2	Semantic vector representation
Database	Supabase PostgreSQL	Profiles, runs, results, evidence and saved profiles
Vector search	pgvector	Cosine similarity retrieval
Deployment	Vercel + Render + Supabase	Hosted frontend, API and database
Testing	pytest + frontend tests	Ranking, API, validation and workflow checks

Architecture flow

Browser UI → FastAPI → Profile extraction → Embedding service → pgvector candidate retrieval → Hybrid ranking → Evidence-grounded explanation → Supabase persistence → Results UI

Gemini should not be the sole ranking authority. Use deterministic, testable scoring over semantic and structured features; use the LLM for extraction and concise explanations. Cache embeddings so the same profile is not repeatedly encoded.

5. AI/ML methodology and ranking

Use Sentence Transformers to encode a candidate profile and the reference profile into vectors. Retrieve semantically similar candidates with cosine similarity. Then re-rank using explicit professional-context features.

Proposed score: Final Score = 0.45S + 0.20R + 0.15E + 0.10N + 0.10L

Component	Weight	Definition
S — Semantic similarity	45%	Cosine similarity between reference and candidate embeddings, scaled to 0–100
R — Role similarity	20%	Similarity of role family and seniority, based on structured categories
E — Education relevance	15%	Strength of school-education indicators and relevant sector experience
N — Network experience	10%	Evidence of work across multiple schools, networks or institutions
L — Location relevance	10%	Geographic proximity or same-region relevance where location is known

Normalise each component to 0–100 before applying weights. If a feature is missing, do not silently treat it as a negative. Either renormalise over available components or mark the score as lower-confidence and disclose which features were unavailable.

Suggested relevance bands

Score	Label	Suggested handling
80–100	High	Prioritise for review
60–79	Medium	Review supporting evidence
Below 60	Low	Keep lower in results or filter out

Explainability requirements

• Show a short explanation tied to observed candidate evidence, not generic praise.

• Display the source/evidence snippets that support key indicators.

• Separate verified facts from model-inferred similarity.

• Never infer a candidate's connections or relationships unless the data explicitly supports them.

• Show 'Not available' for missing fields. Do not invent experience, employers or URLs.

6. Data acquisition, quality and compliance

The assignment explicitly requires authorised access and forbids bypassing LinkedIn authentication, CAPTCHA, access controls or other security mechanisms. Build a compliant ingestion layer with multiple source adapters.

Source method	Use	Controls
User-provided profile text	Reference profile or candidate data supplied by user	Record user-provided source type and import time
CSV import	Curated candidate dataset	Validate schema, provenance and duplicate records
Public organisation pages	School leadership/team pages, education conference speaker pages, consultancy sites	Check terms, permitted use, source URL and retrieval date
Permitted APIs/licensed data	Only when access and intended use are authorised	Store provider, permission basis and limitations
Clearly labelled test dataset	Fallback when 20 genuine records cannot be sourced in time	Mark every row as synthetic/test; never represent as real people

Data quality rules

• Every profile should have a stable internal ID and a provenance record.

• Deduplicate by source identifier where permitted, otherwise by cautious name + organisation matching.

• Track verification status: verified, unverified or test data.

• Track retrieval date and source URL for each evidence item.

• Do not fabricate LinkedIn URLs or claim that a public profile has been verified if it has not.

• Do not expose personal contact details unless the data is lawfully available and necessary.

For the assignment demonstration, aim for 20–50 genuine, relevant, traceable profiles. If a full real dataset is not feasible, use an explicitly identified test dataset and clearly explain the limitation; do not label synthetic records as real.

7. Database design

Table	Key fields / purpose
users	id, name, email, created_at
profiles	id, name, profile_url, role, organisation, location, experience_years, summary, verification_status
profile_evidence	id, profile_id, evidence_text, source_url, source_type, retrieved_at
profile_embeddings	profile_id, embedding vector, model_name, updated_at
discovery_runs	id, user_id, reference_profile_id, status, created_at, parameters
discovery_results	id, run_id, candidate_id, semantic_score, role_score, education_score, network_score, location_score, final_score, explanation
saved_profiles	id, user_id, profile_id, saved_at
data_sources	id, source_name, source_type, permission_status, notes

Database implementation notes

• Enable pgvector in Supabase and create a vector column with the dimension matching the selected embedding model.

• Index vectors using an appropriate pgvector index after testing query volume and recall.

• Keep source evidence separate from the profile's summarised fields.

• Use Supabase Row Level Security if user-specific saved profiles or discovery history are enabled.

• Store API secrets only in backend environment variables.

8. REST API design

Method + endpoint	Purpose
GET /health	Backend health check
POST /api/profiles/analyze	Analyse pasted reference profile text
POST /api/profiles/import	Import and validate CSV records
GET /api/profiles	List/search profiles with pagination
GET /api/profiles/{profile_id}	Profile detail and evidence
POST /api/discovery/run	Start discovery for a reference profile
GET /api/discovery/{run_id}	Get discovery run status and metadata
GET /api/discovery/{run_id}/results	Get ranked candidate results
POST /api/profiles/{profile_id}/save	Save a profile to shortlist
GET /api/analytics/overview	Dashboard metrics
GET /api/discovery/{run_id}/export	Export ranked results as CSV

Use request/response schemas, pagination, consistent error responses, input-size limits, rate limits and structured logging. Add OpenAPI docs through FastAPI's /docs endpoint.

9. Frontend screens and desktop design

Screen	Required UI
Dashboard	KPI cards, recent discovery runs, profile count, new discovery CTA
New Discovery	Profile URL/text input, dataset selection, CSV import, validation preview
Profile Analysis	Extracted fields, editable corrections, missing-data indicators
Discovery Results	Ranked table/cards, relevance score, explanation, source, filters
Profile Details	Role, organisation, location, evidence, source links and match breakdown
Network Analytics	Role distribution, locations, experience segments and score distribution
Saved Profiles	Shortlist, notes, remove/save actions and CSV export
Data Sources	Import history, source types, permission/verification status
Settings	API configuration status, workspace and account preferences

Visual design system

Element	Recommendation
Layout	Desktop-first 1440 × 900; fixed 240–260 px sidebar and top bar
Background	White / very light grey
Sidebar	Deep navy
Primary accent	Blue
Typography	Inter

Element	Recommendation
Components	Consistent cards, tables, badges, dialogs and toasts
Interaction	Loading, empty, success and error states for every workflow
Accessibility	Keyboard focus, contrast, labels and responsive layout

Avoid a generic template appearance. Prioritise a clear discovery action, readable candidate table and visible evidence. Every button shown in the demo must perform a real action or be clearly disabled with an explanation.

10. Recommended repository structure

```
educonnect-ai/ ■■■ frontend/ ■ ■■■ src/ ■ ■ ■■■
components/{layout,dashboard,discovery,profiles,charts}/ ■ ■ ■■■
pages/{Dashboard,NewDiscovery,Results,ProfileDetails,Analytics,DataSources}.tsx ■ ■ ■■■ services/ ■
■ ■■■ hooks/ ■ ■ ■■■ types/ ■ ■ ■■■ App.tsx ■ ■■■ package.json ■■■ backend/ ■ ■■■ app/ ■ ■
■■■ api/{profiles,discovery,analytics}.py ■ ■ ■■■ core/config.py ■ ■ ■■■
services/{profile_extractor,embedding_service,discovery_engine,ranking_engine}.py ■ ■ ■■■ models/ ■
■ ■■■ schemas/ ■ ■ ■■■ main.py ■ ■■■ requirements.txt ■■■ database/{schema.sql,seed.sql} ■■■
data/{sample_profiles.csv,README.md} ■■■ docs/{architecture.md,technical-note.md,demo-script.md}
■■■ .gitignore ■■■ README.md
```

11. Antigravity implementation plan

Build in controlled phases. After each phase, run the app, test the work, fix errors and only then proceed. Do not ask the coding agent to generate the entire system in one pass.

Phase	Build tasks	Exit criteria
1. Scaffold + UI	React/Vite/TypeScript, Tailwind, sidebar, dashboard, forms, result states	Frontend runs; navigation works; no dead buttons
2. Backend + DB	FastAPI, Supabase, SQL schema, Pydantic models, profile CRUD	Health endpoint and database CRUD tested
3. Profile analysis	Gemini extraction, structured JSON, validation, missing fields	Valid and invalid profile inputs handled
4. Embeddings + retrieval	Sentence Transformers, vector storage, pgvector search	Relevant candidates retrieved from seeded dataset
5. Hybrid ranking	Scoring components, normalisation, explanations, deduplication	Unit tests verify score calculations and ordering
6. Results + analytics	Ranked results, filters, details, shortlist, CSV export	End-to-end discovery works with 20+ records
7. Deployment + demo	Vercel, Render, Supabase, environment setup, docs and recording	Live workflow verified from deployed URL

Master prompt for Antigravity

```
Act as a senior full-stack engineer, AI/ML engineer, product designer and software architect. Build
EduConnect AI, an AI-powered school influencer discovery MVP for the EDXSO AI Intern Assignment.
Stack: React + Vite + TypeScript + Tailwind + shadcn/ui; Python FastAPI + Pydantic; Gemini for
structured extraction and explanations; Sentence Transformers all-MiniLM-L6-v2 for embeddings;
Supabase PostgreSQL + pgvector; Vercel frontend and Render backend. The workflow is: reference
profile input -> structured AI analysis -> embedding generation -> authorised candidate retrieval ->
hybrid relevance ranking -> evidence-grounded explanations -> results, shortlist and CSV export.
Support profile text, CSV import and profile URL as a reference identifier. Never assume a LinkedIn
URL can be scraped. Never bypass authentication, CAPTCHA or access controls. Use only authorised
sources. Every candidate must have provenance, source URL where available, retrieval date and
verification status. Do not fabricate real people or claim synthetic profiles are real. Use ranking
weights: 45% semantic similarity, 20% role similarity, 15% school education relevance, 10%
multi-school/network experience, 10% geographic relevance. Normalise components and handle missing
data transparently. Gemini must not invent evidence. Build Dashboard, New Discovery, Profile
Analysis, Results, Profile Details, Analytics, Saved Profiles, Data Sources and Settings. Make all
```

```
visible controls functional. Include tests, README, .env.example, SQL schema, technical note and demo script. Work phase by phase. Start by inspecting the workspace and implementing Phase 1 only: project scaffold and polished desktop-first dashboard UI. Run it and report exact files and commands. Do not proceed to later phases until Phase 1 is tested.
```

Use the master prompt to establish the project, then issue a separate focused prompt for each phase. Ask Antigravity to inspect existing files before modifying them and to report commands, changed files and test results.

12. Environment configuration and setup

Suggested environment variables

```
# backend/.env (never commit the real file) GEMINI_API_KEY=your_gemini_api_key
SUPABASE_URL=your_supabase_project_url SUPABASE_SERVICE_ROLE_KEY=your_server_only_service_role_key
SUPABASE_ANON_KEY=your_anon_key_if_needed DATABASE_URL=your_postgres_connection_string
FRONTEND_ORIGIN=http://localhost:5173 EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

Local setup outline

Frontend: cd frontend → npm install → npm run dev

Backend: create and activate a Python virtual environment → pip install -r requirements.txt → uvicorn app.main:app --reload

Database: create Supabase project → enable pgvector → run database/schema.sql → import a permitted seed dataset.

Exact commands may need small adjustments based on the generated package and module structure. Document the commands that actually work in README.md.

13. Testing and acceptance criteria

Test area	Acceptance criteria
Input validation	Malformed URLs, empty text and invalid CSV rows return clear errors
Extraction	Structured fields validate; missing values remain missing
Embeddings	Vectors generated with expected dimensions and model identifier stored
Retrieval	Semantic search returns plausible school-ecosystem candidates
Ranking	Weights, normalisation and tie ordering covered by unit tests
Evidence	Every displayed match reason is supported by stored evidence
Dataset	At least 20 relevant candidates returned for the chosen test reference profile
Traceability	Source, access method, retrieval date and verification status visible
UI	Loading, empty, error and success states work; filters and export work
Deployment	Public URL, backend health and database connectivity verified

Do not hardcode '1,250 profiles', '48 discovery runs' or sample scores into dashboard metrics. Dashboard metrics must be derived from actual database records. Demo examples should be visibly labelled as illustrative.

14. Deployment plan

Service	Deployment steps
Supabase	Create project, enable pgvector, run schema, configure RLS and secrets
Render	Deploy FastAPI backend, set Python build/start commands and environment variables

Service	Deployment steps
Vercel	Deploy frontend, set backend API base URL and allowed origin
Integration	Configure CORS, test auth if enabled, verify all API calls and database queries
Final verification	Open deployed URL in private/incognito session, run a complete discovery and export results

Keep service-role credentials on the backend only. Never put Gemini keys or Supabase service-role keys into frontend variables. Deployment is complete only after a real end-to-end test.

15. Demo video plan (5–10 minutes)

Time	What to demonstrate
0:00–0:45	Problem statement and EduConnect AI introduction
0:45–1:45	Enter reference profile or choose test reference record
1:45–2:45	Show AI-extracted professional attributes and evidence
2:45–4:00	Explain embeddings, candidate retrieval and hybrid ranking
4:00–6:00	Show at least 20 relevant results, scores and match explanations
6:00–7:00	Open profile evidence, apply filters, save and export CSV
7:00–8:30	Show architecture, data sourcing, limitations and compliance

Keep your face visible if the assignment instructions require it. Do not imply that simulated data is real. State clearly which sources and access methods were used.

16. Technical note outline

• Problem and product overview.

• System architecture and major components.

• Data retrieval approach and source permissions.

• Profile extraction schema and validation.

• Embedding model and semantic similarity method.

• Hybrid ranking formula, weights and missing-data handling.

• Explanation generation and evidence traceability.

• Dataset size, composition, verification and known limitations.

• Privacy, platform compliance and production access path.

• Local setup, environment variables, database migration and deployment.

17. Risks, limitations and production path

Risk	Mitigation
LinkedIn profile cannot be fetched directly	Treat URL as identifier; request pasted text or use permitted source/API
Small or biased dataset	Document coverage; broaden authorised sources; evaluate relevance manually

Risk	Mitigation
LLM hallucination	Use schema validation and evidence-grounded explanations; never invent fields
Embedding similarity misses seniority/context	Use hybrid re-ranking and role/sector features
Missing candidate data	Show unavailable and lower confidence; avoid false negatives
Sensitive personal data	Minimise collection, document purpose, secure access and retention
Deployment secrets leak	Backend-only secrets, .gitignore and secret rotation

A compliant production path could use a licensed/authorised professional data provider or permitted platform API, with contractual review, source provenance and user-visible disclosure. Do not promise LinkedIn integration until access is confirmed.

18. Final submission checklist

• Working deployed MVP with public or reviewer-accessible URL.

• GitHub repository with source, README, dependencies, .env.example and SQL schema.

• At least 20 relevant candidate profiles in the demonstrated dataset, with honest source labels.

• AI/ML similarity or ranking method explained and tested.

• Ranked results with match scores, evidence and short explanations.

• 5–10 minute demo video.

• Technical note covering architecture, data retrieval, AI method, ranking, limitations and setup.

• Final check: no secrets committed, no fake profiles represented as real, no dead buttons, no unverified claims.

Recommended final positioning: **EduConnect AI — an explainable, evidence-backed discovery engine for the school education ecosystem.**